C# Programming for Indian Students — Chapter 7: Methods

Nov 3, 2026 · @Vikash

7.1 What you will learn

In Chapter 6 your programs learned to repeat. In this chapter they learn to be organised. Look at any app you use every day: the ATM does not keep one giant list of instructions, it has separate jobs such as *show the menu*, *check the balance*, *withdraw money*. Each job has a name, and the program simply calls the name whenever the job is needed. In C# such a named block of code is called a **method**. By the end of this chapter you will be able to:

- write a method once and use it as many times as you like;
- send information into a method with **parameters** and get an answer back with a **return value**;
- split the long programs of Chapters 4 to 6 into small, readable methods;
- explain why changing a parameter inside a method does not change the original variable, and when `ref` and `out` do;
- use optional parameters, named arguments and **overloading**;
- write a simple recursive method and read the compiler messages for the most common method mistakes;
- build a Mini ATM in which every job is its own method.

You have already used methods. `Console.WriteLine(...)`, `Console.ReadLine()`, `int.TryParse(...)` and `Math.Sqrt(...)` are all methods written by Microsoft. In this chapter you write your own.

Where methods go in these programs. All programs in this book use top-level statements, as in Chapters 3 to 6. In such a program you write your own methods **below** the statements that use them, in the same `Program.cs` file. (Technically the compiler turns them into *local functions* of the hidden `Main` method. The rules you learn here are the same ones you will use later when methods live inside classes. The one place where the difference shows is overloading, section 7.8.) Every program in this chapter was compiled and run on the .NET 10 SDK with the sample input shown.

Vikash Kumar

7.2 Your first method

A method is a named block of code. You **define** it once, then **call** it whenever you need that job done. The simplest method takes nothing and gives nothing back:

```
void MethodName()
{
    // statements that do the job
}
```

- `void` means *this method does not return a value*. It only does something, such as printing.
- `MethodName` is the name. Use **PascalCase** (every word starts with a capital) and a verb: `PrintBanner`, `ShowMenu`, `CalculateBill`.
- The empty brackets `()` mean *no parameters yet*; they are still required.
- The statements go inside the braces `{ }`, the **body** of the method.

You call the method by writing its name followed by brackets and a semicolon: `PrintBanner();`.

Program 7.1: Your first method

```
Console.WriteLine("Welcome to the college fest!");
PrintBanner();
Console.WriteLine("Registration is open.");
PrintBanner();
Console.WriteLine("Thank you for visiting.");
PrintBanner();

void PrintBanner()
{
    Console.WriteLine("==============================");
}
```

Output

```
Welcome to the college fest!
==============================
Registration is open.
==============================
Thank you for visiting.
==============================
```

Vikash Kumar

How a call works. When the program reaches `PrintBanner();` it *pauses* the current line, jumps into the body of `PrintBanner`, runs every statement there, and then *returns* to the line just after the call. The banner line is written once in the program but runs three times. If the college wants a different banner tomorrow, you change one line instead of three. This is the first big reason for methods: **write once, use many times**.

Remember. A method does not run when it is defined. It runs only when it is called. A defined method that nobody calls does nothing.

7.3 Parameters and arguments

Most jobs need some information. A greeting needs the student's name; a table needs the number. You pass information into a method through **parameters**, which are variables written inside the brackets of the definition.

```
void Greet(string name, string college)
```

- A **parameter** is the variable in the definition: `name`, `college`. Each one has a type, and several are separated by commas.
- An **argument** is the actual value you give at the call: `Greet("Aarav", "Delhi University");`.
- Arguments are matched to parameters **by position**: the first argument goes to the first parameter, the second to the second.
- The number and the types of the arguments must match the parameters, otherwise the compiler reports an error (section 7.11).

Program 7.2: Parameters and arguments

Vikash Kumar

```
Greet("Aarav", "Delhi University");
Greet("Diya", "Pune University");

PrintTable(5, 5);
PrintTable(12, 3);

void Greet(string name, string college)
{
    Console.WriteLine($"Welcome {name} to {college}!");
}

void PrintTable(int number, int rounds)
{
    for (int i = 1; i <= rounds; i++)
    {
        Console.WriteLine($"{number} x {i} = {number * i}");
    }
}
```

Output

```
Welcome Aarav to Delhi University!
Welcome Diya to Pune University!
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
12 x 1 = 12
12 x 2 = 24
12 x 3 = 36
```

Vikash Kumar

What to notice. `PrintTable` contains the `for` loop of Chapter 6. The loop is now written once and works for any number and any count. The parameters `number` and `rounds` exist only while the method runs. Inside the method they behave like ordinary variables.

7.4 Return values

Sometimes you want the method to give you an answer, such as the square of a number or the grade for some marks. Replace `void` with the **type of the answer** and finish the method with a `return` statement:

```
int Square(int number)
{
    return number * number;
}
```

- The word before the name is the **return type**: `int`, `bool`, `string`, `decimal` and so on.
- `return value;` ends the method immediately and sends the value back to the caller.
- The call itself becomes a value, so you can store it (`int sq = Square(9);`), print it, or use it inside a bigger expression.
- Every possible path through the method must end in a `return`. A method can have several `return` statements, for example inside `if` blocks.

Program 7.3: Methods that return a value

```
int sq = Square(9);
Console.WriteLine($"Square of 9 = {sq}");

int bigger = Max(42, 17);
Console.WriteLine($"Larger of 42 and 17 = {bigger}");

Console.WriteLine($"Is 28 even? {IsEven(28)}");
Console.WriteLine($"Is 15 even? {IsEven(15)}");

int[] marks = { 91, 78, 64, 45, 32 };
foreach (int m in marks)
{
    Console.WriteLine($"{m} marks -> grade {GetGrade(m)}");
}

int Square(int number)
{
    return number * number;
}

int Max(int a, int b)
{
    if (a > b)
    {
        return a;
    }
    return b;
}

bool IsEven(int number)
{
```

Vikash Kumar

```
    return number % 2 == 0;
}

string GetGrade(int marks)
{
    if (marks >= 90) return "A";
    if (marks >= 75) return "B";
    if (marks >= 60) return "C";
    if (marks >= 40) return "D";
    return "F";
}
```

Output

```
Square of 9 = 81
Larger of 42 and 17 = 42
Is 28 even? True
Is 15 even? False
91 marks -> grade A
78 marks -> grade B
64 marks -> grade C
45 marks -> grade D
32 marks -> grade F
```

Design tips.

- A method that answers a yes/no question (`IsEven` , `IsPrime`) returns `bool` and is named like a question.
- A method should do **one job**. `GetGrade` only decides the grade; it does not read marks and it does not print. The caller decides what to do with the answer. Such methods are easy to test and to reuse.
- The array `{ 91, 78, ... }` is only a small list of values for `foreach` . Arrays are taught properly in Chapter 8.

7.5 Splitting a program into methods

The real power of methods is **decomposition**: breaking one long program into small jobs, each with a clear name. Compare the electricity bill of Program 5.4, which was one long block, with the same program rewritten with methods. The program now reads like a story: *read the units, calculate the bill, print the bill*.

Program 7.4: Electricity bill with methods (slab rates are example values, not a real tariff)

Vikash Kumar

```
int units = ReadUnits();
decimal bill = CalculateBill(units);
PrintBill(units, bill);

int ReadUnits()
{
    while (true)
    {
        Console.Write("Units consumed: ");
        string text = Console.ReadLine() ?? "";

        if (int.TryParse(text, out int units) && units >= 0)
        {
            return units;
        }
        Console.WriteLine("Please type a whole number, zero or more.");
    }
}

decimal CalculateBill(int units)
{
    if (units <= 100)
    {
        return units * 3.00m;
    }
    if (units <= 200)
    {
        return 100 * 3.00m + (units - 100) * 4.50m;
    }
    if (units <= 300)
    {
        return 100 * 3.00m + 100 * 4.50m + (units - 200) * 6.00m;
    }
    return 100 * 3.00m + 100 * 4.50m + 100 * 6.00m + (units - 300) * 7.50m;
}

void PrintBill(int units, decimal bill)
{
    Console.WriteLine("----- Electricity Bill -----");
    Console.WriteLine($"Units consumed : {units}");
    Console.WriteLine($"Bill amount    : Rs. {bill:F2}");
}
```

Output (typed input: `abc`, then `250`)

Vikash Kumar

```
Units consumed: abc
Please type a whole number, zero or more.
Units consumed: 250
----- Electricity Bill -----
Units consumed : 250
Bill amount    : Rs. 1050.00
```

Why this is better.

- Each method has one job and a name that says what it is. The first three lines of the program tell the whole story.
- The loop that asks again until the input is valid (from Section 6.5) is hidden inside `ReadUnits`. Any other program can reuse it.
- `CalculateBill` neither reads nor prints. To test it you only need to call `CalculateBill(250)` and check that the answer is `1050.00`. The bill for 250 units is 100 x 3.00 + 100 x 4.50 + 50 x 6.00 = 300 + 450 + 300 = 1050.
- Several `return` statements replace the `else if` ladder: as soon as one `return` runs, the method ends, so no `else` is needed.
- Note that the variable `units` appears in three places. They are three different variables, each local to its own method. That is the topic of the next section.

7.6 Parameters are copies, and scope

Two rules explain most method surprises.

Rule 1: pass by value. When you call a method with a variable, C# gives the method a **copy** of the value. The method can change its copy, but the original variable in the caller stays the same.

Rule 2: scope. A variable declared inside a method (including its parameters) is **local**. It exists only while the method runs and cannot be seen from outside. Another method cannot use it, even if it has the same name.

Program 7.5: Parameters are copies

Vikash Kumar

```
int marks = 70;
Console.WriteLine($"Before AddBonusWrong : marks = {marks}");
AddBonusWrong(marks);
Console.WriteLine($"After AddBonusWrong  : marks = {marks}");

marks = AddBonus(marks);
Console.WriteLine($"After AddBonus       : marks = {marks}");

void AddBonusWrong(int marks)
{
    marks += 5;
    Console.WriteLine($"Inside AddBonusWrong : marks = {marks}");
}

int AddBonus(int marks)
{
    return marks + 5;
}
```

Output

```
Before AddBonusWrong : marks = 70
Inside AddBonusWrong : marks = 75
After AddBonusWrong  : marks = 70
After AddBonus       : marks = 75
```

Explanation. `AddBonusWrong` receives a copy of 70, adds 5 to the copy and prints 75, but the original `marks` in the main program is still 70. The method does not return anything, so the work is lost. `AddBonus` returns the new value, and the caller stores it with `marks = AddBonus(marks);`. **The usual way to get a changed value out of a method is the return value.** When a method must send back more than one value, or must change the caller's variable, C# offers `ref` and `out` (section 7.9).

7.7 Optional parameters and named arguments

Some parameters almost always have the same value. GST is usually 18 percent; most students do not live in the hostel. Give such a parameter a **default value** and the caller may leave it out. A parameter with a default value is called an **optional parameter**.

```
decimal CalculateFee(decimal amount, decimal gstPercent = 18m, bool hostel =
false)
```

- Optional parameters must come **after** all the required ones.
- When you skip an argument, the default is used.

Vikash Kumar

- With a **named argument** you write the parameter name and a colon (`hostel: true`). Then the position no longer matters, and you can skip optional parameters in the middle. Named arguments also make a call easier to read.

Program 7.6: Optional parameters and named arguments

```
decimal fee1 = CalculateFee(50000m);
decimal fee2 = CalculateFee(50000m, 5m);
decimal fee3 = CalculateFee(50000m, hostel: true);
decimal fee4 = CalculateFee(hostel: true, amount: 50000m, gstPercent: 0m);

Console.WriteLine($"Fee 1 (default GST 18%)      : Rs. {fee1:F2}");
Console.WriteLine($"Fee 2 (GST 5%)               : Rs. {fee2:F2}");
Console.WriteLine($"Fee 3 (default GST, hostel)  : Rs. {fee3:F2}");
Console.WriteLine($"Fee 4 (no GST, hostel)       : Rs. {fee4:F2}");

decimal CalculateFee(decimal amount, decimal gstPercent = 18m, bool hostel =
false)
{
    decimal total = amount + amount * gstPercent / 100;
    if (hostel)
    {
        total += 20000m;
    }
    return total;
}
```

Output

```
Fee 1 (default GST 18%)     : Rs. 59000.00
Fee 2 (GST 5%)              : Rs. 52500.00
Fee 3 (default GST, hostel)  : Rs. 79000.00
Fee 4 (no GST, hostel)      : Rs. 70000.00
```

The fee, GST rate and hostel charge are example values only. In the third call, `hostel: true` skips `gstPercent`, which keeps its default of 18.

7.8 Method overloading

Overloading means writing several methods with the **same name** but **different parameters** (a different number of parameters, or different types). When you call the method, the compiler looks at your arguments and picks the matching version. You have already used overloading: `Console.WriteLine` has versions for strings, integers, decimals and many more.

Vikash Kumar

Two methods are different overloads if their parameter lists differ. A different **return type alone** is not enough.

A special rule for this chapter. Overloaded methods must live inside a class, because the compiler does not allow two top-level methods with the same name (see the error in section 7.11). Until classes are explained properly in a later chapter, put them in a `static class` at the **bottom** of `Program.cs`, and call them as `ClassName.MethodName(...)`. Read `MathHelper.Add` as *the Add method that belongs to MathHelper*. The word `public` makes the method visible outside the class, and `static` lets you call it without creating an object.

Program 7.7: Method overloading

```
Console.WriteLine($"Add(2, 3)        = {MathHelper.Add(2, 3)}");
Console.WriteLine($"Add(2, 3, 4)     = {MathHelper.Add(2, 3, 4)}");
Console.WriteLine($"Add(2.5, 3.5)    = {MathHelper.Add(2.5, 3.5)}");

Console.WriteLine($"Area(5)          = {MathHelper.Area(5)}");
Console.WriteLine($"Area(5, 3)       = {MathHelper.Area(5, 3)}");
Console.WriteLine($"Area(2.5, 4.0)   = {MathHelper.Area(2.5, 4.0)}");

static class MathHelper
{
    public static int Add(int a, int b)
    {
        return a + b;
    }

    public static int Add(int a, int b, int c)
    {
        return a + b + c;
    }

    public static double Add(double a, double b)
    {
        return a + b;
    }

    public static int Area(int side)
    {
        return side * side;
    }

    public static int Area(int length, int breadth)
    {
        return length * breadth;
```

Vikash Kumar

```
    }

    public static double Area(double length, double breadth)
    {
        return length * breadth;
    }
}
```

Output

```
Add(2, 3)        = 5
Add(2, 3, 4)     = 9
Add(2.5, 3.5)    = 6
Area(5)          = 25
Area(5, 3)       = 15
Area(2.5, 4.0)   = 10
```

How the compiler chooses. `Add(2, 3)` has two `int` arguments, so the `int, int` version runs. `Add(2, 3, 4)` has three arguments, so the three-parameter version runs. `Add(2.5, 3.5)` has two `double` arguments, so the `double` version runs, and the answer prints as `6` because a `double` result of 6.0 is shown without a decimal point by default. Overloading lets callers use one easy-to-remember name for one idea (adding, area) instead of `AddInts`, `AddDoubles`, `AddThree`.

7.9 ref and out parameters

A method can return only one value. Two situations need more:

1. The method must **change the caller's variable**, such as swapping two numbers.
2. The method must **send back more than one result**, such as a quotient and a remainder, or a *yes/no* together with a value (exactly what `int.TryParse` does).

C# provides two keywords for this. Both pass the variable itself, not a copy, and both must be written at the definition **and** at the call.

Keyword	The variable must be set before the call?	The method must set it before returning?	Typical use
`ref`	Yes	No (it may only read or change it)	Change the caller's variable, such as `Swap`
`out`	No	Yes	Send back an extra result, such as `TryParse`

Vikash Kumar

Program 7.8: ref and out

```
int a = 10;
int b = 20;
Console.WriteLine($"Before swap : a = {a}, b = {b}");
Swap(ref a, ref b);
Console.WriteLine($"After swap  : a = {a}, b = {b}");

string[] inputs = { "21", "abc", "-4" };
foreach (string text in inputs)
{
    if (TryReadAge(text, out int age))
    {
        Console.WriteLine($"\"{text}\" is a valid age: {age}");
    }
    else
    {
        Console.WriteLine($"\"{text}\" is not a valid age.");
    }
}

Divide(47, 5, out int quotient, out int remainder);
Console.WriteLine($"47 / 5 : quotient = {quotient}, remainder =
{remainder}");

void Swap(ref int first, ref int second)
{
    int temp = first;
    first = second;
    second = temp;
}

bool TryReadAge(string text, out int age)
{
    if (int.TryParse(text, out age) && age >= 0 && age <= 120)
    {
        return true;
    }
    age = 0;
    return false;
}

void Divide(int dividend, int divisor, out int quotient, out int remainder)
```

Vikash Kumar

```
{
    quotient = dividend / divisor;
    remainder = dividend % divisor;
}
```

Output

```
Before swap : a = 10, b = 20
After swap  : a = 20, b = 10
"21" is a valid age: 21
"abc" is not a valid age.
"-4" is not a valid age.
47 / 5 : quotient = 9, remainder = 2
```

Explanation.

- `Swap(ref a, ref b)` gives the method the variables themselves. Swapping `first` and `second` really swaps `a` and `b`. Compare Program 7.5, where a plain parameter was only a copy.
- `TryReadAge` follows the same pattern as `int.TryParse`: it returns `true` or `false` for success, and sends the age out through the `out` parameter. In the failure case it still must assign `age` (here `0`), because an `out` parameter has to be assigned on every path.
- `out int quotient` in the call declares the variable at the same time. This is the style you used with `TryParse` in Chapter 5.
- Use `ref` and `out` sparingly. For one answer, a return value is clearer.

7.10 Short methods and recursion

Expression-bodied methods. When the whole method is a single expression, you can replace the braces and `return` with the arrow `=>`. It is only a shorter way to write the same method. Methods can also call other methods, which is how big programs are built from small pieces.

Program 7.9: Short methods and methods that call methods

Vikash Kumar

```
Console.WriteLine($"Cube of 4          = {Cube(4)}");
Console.WriteLine($"Is 2028 a leap year? {IsLeapYear(2028)}");
Console.WriteLine($"Is 2100 a leap year? {IsLeapYear(2100)}");
Console.WriteLine($"Days in Feb 2028   = {DaysInFebruary(2028)}");
Console.WriteLine($"Days in Feb 2100   = {DaysInFebruary(2100)}");

int Cube(int n) => n * n * n;

bool IsLeapYear(int year) => (year % 4 == 0 && year % 100 != 0) || year % 400
== 0;

int DaysInFebruary(int year) => IsLeapYear(year) ? 29 : 28;
```

Output

```
Cube of 4          = 64
Is 2028 a leap year? True
Is 2100 a leap year? False
Days in Feb 2028   = 29
Days in Feb 2100   = 28
```

`DaysInFebruary` does not repeat the leap-year rule. It calls `IsLeapYear`, so the rule lives in exactly one place.

Recursion. A method may even call **itself**. This is called **recursion**. A recursive method needs two parts:

- a **base case**, a small input the method can answer directly without calling itself;
- a **recursive case**, which calls the method again with a **smaller** input, moving closer to the base case.

Without a base case the method never stops (see section 7.11).

Program 7.10: Recursion

```
CountDown(5);
Console.WriteLine($"5! = {Factorial(5)}");
Console.WriteLine($"Sum of digits of 4821 = {SumOfDigits(4821)}");

void CountDown(int n)
{
    if (n == 0)
    {
        Console.WriteLine("Lift off!");
        return;
    }
    Console.WriteLine(n);
```

Vikash Kumar

```
    CountDown(n - 1);
}

long Factorial(int n)
{
    if (n <= 1)
    {
        return 1;
    }
    return n * Factorial(n - 1);
}

int SumOfDigits(int n)
{
    if (n < 10)
    {
        return n;
    }
    return n % 10 + SumOfDigits(n / 10);
}
```

Output

```
5
4
3
2
1
Lift off!
5! = 120
Sum of digits of 4821 = 15
```

How `Factorial(3)` works. Each call waits for the next one to finish:

```
Factorial(3) = 3 * Factorial(2)
                   Factorial(2) = 2 * Factorial(1)
                                      Factorial(1) = 1      <- base case
                   Factorial(2) = 2 * 1 = 2
Factorial(3) = 3 * 2 = 6
```

Anything a recursive method does can also be done with a loop (Question 10 asks for both). Use recursion when it makes the idea clearer; use a loop when the number of rounds can be very large.

Vikash Kumar

7.11 Dry run and common errors

Dry run. Trace a program on paper, keeping track of which method is running. Take Program 7.11:

```
int result = Double(Add(2, 3)) + Add(1, 1);
Console.WriteLine($"result = {result}");

int Add(int a, int b)
{
    int sum = a + b;
    Console.WriteLine($"Add({a}, {b}) returns {sum}");
    return sum;
}

int Double(int n)
{
    int twice = n * 2;
    Console.WriteLine($"Double({n}) returns {twice}");
    return twice;
}
```

The expression is evaluated from left to right, and a call's arguments are evaluated before the call itself, so the inner `Add(2, 3)` runs first:

Step	What happens	Printed
1	`Add(2, 3)` runs: `sum = 5`, returns 5	`Add(2, 3) returns 5`
2	`Double(5)` runs with that 5: `twice = 10`, returns 10	`Double(5) returns 10`
3	`Add(1, 1)` runs: `sum = 2`, returns 2	`Add(1, 1) returns 2`
4	`result = 10 + 2 = 12`	`result = 12`

Output

```
Add(2, 3) returns 5
Double(5) returns 10
Add(1, 1) returns 2
result = 12
```

Common errors. The messages below are the real build output of the .NET 10 SDK.

Virash Kumar

Mistake	What you see	The fix
A path with no `return` (for example only `if (m >= 40) return "Pass";`)	`error CS0161: 'Grade(int)': not all code paths return a value`	Make sure every path ends in a `return`, such as a final `return "Fail";`
Calling `Greet();` when the method needs a `name`	`error CS7036: There is no argument given that corresponds to the required parameter 'name' of 'Greet(string)'`	Pass every required argument: `Greet("Aarav");`
Passing text where a number is needed: `Square("5")`	`error CS1503: Argument 1: cannot convert from 'string' to 'int'`	Pass the right type: `Square(5)`
Writing `return 5;` inside a `void` method	`error CS0127: Since 'Show()' returns void, a return keyword must not be followed by an object expression`	Change the return type to `int`, or remove the value
Using the result of a `void` method: `int x = Show();`	`error CS0029: Cannot implicitly convert type 'void' to 'int'`	Give `Show` a return type, or do not use its result
Using a method's local variable outside it: `Console.WriteLine(sum);`	`error CS0103: The name 'sum' does not exist in the current context`	Return the value from the method and store it in the caller
An `out` parameter never assigned	`error CS0177: The out parameter 'value' must be assigned to before control leaves the current method`	Assign the `out` parameter on every path
Passing a plain variable to a `ref` parameter: `Inc(a);`	`error CS1620: Argument 1 must be passed with the 'ref' keyword`	Write `Inc(ref a);`

Vikash Kumar

Mistake	What you see	The fix
Two top-level methods with the same name (overloading without a class)	`error CS0128: A local variable or function named 'Convert' is already defined in this scope` and then `error CS1501: No overload for method 'Convert' takes 2 arguments`	Move the overloads into a `static class` (section 7.8)

A run-time error: no base case. The compiler cannot catch a recursive method that never stops. This program compiles without a warning:

```
Console.WriteLine(Count(1));

int Count(int n)
{
    return Count(n + 1);
}
```

When you run it, every call waits for the next one, the calls pile up in memory until the space for them is full, and the program is stopped:

```
Stack overflow.
Repeated 260782 times:
--------------------------------
   at Program.<<Main>$>g__Count|0_0(Int32)
```

The number of repeats differs from computer to computer. The cure is always the same: check that the method has a base case and that every recursive call moves closer to it.

7.12 Mini project: Mini ATM

In Section 6.3 you wrote a bank menu with `do-while`. Now rebuild it the professional way: **every job is a method**, and the top of the program is only the menu loop.

Requirements

1. Start with a balance of Rs. 10,000.
2. Show a menu: check balance, deposit, withdraw, exit. Repeat until the user chooses Exit.

Vikash Kumar

3. Read the menu choice and every amount with input checking. Ask again on a wrong entry.
4. Refuse a withdrawal larger than the balance.

Methods used

Method	Job	How it gives its result
`ShowMenu()`	Print the four options	Nothing (`void`)
`ReadChoice()`	Ask until the user types 1 to 4	Returns the choice (`int`)
`ReadAmount(prompt)`	Ask until the user types an amount greater than zero	Returns the amount (`decimal`)
`ShowBalance(current)`	Print the balance	Nothing (`void`)
`Deposit(ref current, amount)`	Add the amount to the balance	Changes the caller's balance through `ref`
`TryWithdraw(ref current, amount)`	Subtract if enough money is there	Returns `true` or `false` , and changes the balance through `ref`

Program 7.12: Mini ATM

```
decimal balance = 10000m;
bool running = true;

while (running)
{
    ShowMenu();
    int choice = ReadChoice();

    switch (choice)
    {
        case 1:
            ShowBalance(balance);
            break;
        case 2:
            decimal depositAmount = ReadAmount("Amount to deposit: ");
            Deposit(ref balance, depositAmount);
            break;
        case 3:
            decimal withdrawAmount = ReadAmount("Amount to withdraw: ");
            if (!TryWithdraw(ref balance, withdrawAmount))
```

Vikash Kumar

```
            {
                Console.WriteLine("Insufficient balance. Withdrawal
cancelled.");
            }
            break;
        case 4:
            running = false;
            break;
    }
}
Console.WriteLine("Thank you for banking with us. Jai Hind!");

void ShowMenu()
{
    Console.WriteLine();
    Console.WriteLine("===== Mini ATM =====");
    Console.WriteLine("1. Check balance");
    Console.WriteLine("2. Deposit");
    Console.WriteLine("3. Withdraw");
    Console.WriteLine("4. Exit");
}

int ReadChoice()
{
    while (true)
    {
        Console.Write("Choose 1-4: ");
        if (int.TryParse(Console.ReadLine(), out int choice) && choice >= 1
&& choice <= 4)
        {
            return choice;
        }
        Console.WriteLine("Invalid choice. Try again.");
    }
}

decimal ReadAmount(string prompt)
{
    while (true)
    {
        Console.Write(prompt);
        if (decimal.TryParse(Console.ReadLine(), out decimal amount) &&
amount > 0)
        {
            return amount;
        }
```

Vikash Kumar

```
        Console.WriteLine("Please type an amount greater than zero.");
    }
}

void ShowBalance(decimal current)
{
    Console.WriteLine($"Current balance: Rs. {current:F2}");
}

void Deposit(ref decimal current, decimal amount)
{
    current += amount;
    Console.WriteLine($"Deposited Rs. {amount:F2}. New balance: Rs.
{current:F2}");
}

bool TryWithdraw(ref decimal current, decimal amount)
{
    if (amount > current)
    {
        return false;
    }
    current -= amount;
    Console.WriteLine($"Withdrew Rs. {amount:F2}. New balance: Rs.
{current:F2}");
    return true;
}
```

Output (typed input: `1, 2, abc, 2500, 3, 20000, 3, 4000, 9, 1, 4`)

```
===== Mini ATM =====
1. Check balance
2. Deposit
3. Withdraw
4. Exit
Choose 1-4: 1
Current balance: Rs. 10000.00

===== Mini ATM =====
1. Check balance
2. Deposit
3. Withdraw
4. Exit
Choose 1-4: 2
Amount to deposit: abc
Please type an amount greater than zero.
```

Vikash Kumar

```
Amount to deposit: 2500
Deposited Rs. 2500.00. New balance: Rs. 12500.00

===== Mini ATM =====
1. Check balance
2. Deposit
3. Withdraw
4. Exit
Choose 1-4: 3
Amount to withdraw: 20000
Insufficient balance. Withdrawal cancelled.

===== Mini ATM =====
1. Check balance
2. Deposit
3. Withdraw
4. Exit
Choose 1-4: 3
Amount to withdraw: 4000
Withdrew Rs. 4000.00. New balance: Rs. 8500.00

===== Mini ATM =====
1. Check balance
2. Deposit
3. Withdraw
4. Exit
Choose 1-4: 9
Invalid choice. Try again.
Choose 1-4: 1
Current balance: Rs. 8500.00

===== Mini ATM =====
1. Check balance
2. Deposit
3. Withdraw
4. Exit
Choose 1-4: 4
Thank you for banking with us. Jai Hind!
```

Vikash Kumar

Design decisions.

- The main loop is short and reads like the menu itself. Each `case` calls one or two methods.
- Input checking is separated from banking. `ReadChoice` and `ReadAmount` never touch the balance, and `Deposit` and `TryWithdraw` never read the keyboard.

- `TryWithdraw` is named like `TryParse` on purpose: it tries, and tells you with `true` or `false` whether it worked. The caller decides which message to show.
- The balance is a `decimal` (Chapter 3) because it is money.
- `ref decimal current` lets `Deposit` and `TryWithdraw` change the balance of the main program directly. In a later chapter, a *class* will hold the balance and these methods together, and `ref` will no longer be needed.
- Challenge: add a fifth option, *Mini statement*, that shows how many deposits and withdrawals were made. Which new method would you write, and what would it need to know?

7.13 Practice questions with answers

Try every question yourself before reading the answer. The solution programs are in the `Solutions` folder of the chapter in the code repository.

Q1. What is a method? Name two methods you have already used. A method is a named block of code that does one job and can be called again and again. `Console.WriteLine` and `int.TryParse` are two methods you have used.

Q2. What is the difference between a `void` method and a method that returns a value? A `void` method only performs an action, such as printing, and gives nothing back. A method with a return type such as `int` or `bool` ends with `return value;`, and the call can be used as a value.

Q3. What is the difference between a parameter and an argument? A parameter is the variable written in the method's definition (`int number`). An argument is the actual value supplied at the call (`Square(9)`, where 9 is the argument).

Q4. Why does changing a parameter inside a method not change the caller's variable? How can the caller get the new value? By default C# passes a copy of the value (pass by value). The method changes only its copy. The caller can receive the new value through the return value (`marks = AddBonus(marks);`), or the method can use a `ref` or `out` parameter.

Q5. What is overloading? Can two methods differ only in their return type? Overloading means several methods with the same name and different parameter lists (different number or types of parameters). Two methods cannot differ only in their return type, because the compiler chooses the version from the arguments.

Q6. Predict the output.

Vikash Kumar

```
Show(3);
Show(1);

void Show(int n)
{
    for (int i = 1; i <= n; i++)
    {
        Console.Write(i * 2 + " ");
    }
    Console.WriteLine();
}
```

```
2 4 6
2
```

`Show(3)` prints 2, 4, 6 and then a new line. `Show(1)` prints only 2.

Q7. Predict the output.

```
int x = 5;
Change(x);
Console.WriteLine(x);
Console.WriteLine(Change(x));

int Change(int x)
{
    x = x * 3;
    return x + 1;
}
```

```
5
16
```

The first call throws its answer away, and `x` in the main program stays 5. The second call receives 5, makes it 15, returns 16, and `WriteLine` prints 16.

Q8. Predict the output.

Vikash Kumar

```
Console.WriteLine(Mystery(4));
Console.WriteLine(Mystery(1));

int Mystery(int n)
{
    if (n <= 1)
    {
        return 1;
    }
    return n + Mystery(n - 2);
}
```

```
7
1
```

`Mystery(4) = 4 + Mystery(2) = 4 + (2 + Mystery(0)) = 4 + 2 + 1 = 7`. `Mystery(1)` is the base case and returns 1.

Q9. Write a method `IsPrime(int number)` and use it to print all primes up to a limit.

```
Console.Write("Print primes up to: ");
int limit = int.Parse(Console.ReadLine() ?? "");

for (int n = 2; n <= limit; n++)
{
    if (IsPrime(n))
    {
        Console.Write(n + " ");
    }
}
Console.WriteLine();

bool IsPrime(int number)
{
    if (number < 2)
    {
        return false;
    }
    for (int d = 2; d * d <= number; d++)
    {
        if (number % d == 0)
        {
            return false;
```

Vikash Kumar

```
        }
    }
    return true;
}
```

```
Print primes up to: 30
2 3 5 7 11 13 17 19 23 29
```

The method stops testing at the square root of the number (`d * d <= number`): a number that has a divisor larger than its square root must also have a smaller one.

Q10. Write the factorial of a number twice: once with a loop and once with recursion.

```
Console.Write("Enter a number from 0 to 20: ");
int n = int.Parse(Console.ReadLine() ?? "");

Console.WriteLine($"{n}! by loop      = {FactorialLoop(n)}");
Console.WriteLine($"{n}! by recursion = {FactorialRecursive(n)}");

long FactorialLoop(int number)
{
    long result = 1;
    for (int i = 2; i <= number; i++)
    {
        result *= i;
    }
    return result;
}

long FactorialRecursive(int number)
{
    return number <= 1 ? 1 : number * FactorialRecursive(number - 1);
}
```

```
Enter a number from 0 to 20: 10
10! by loop      = 3628800
10! by recursion = 3628800
```

The return type is `long` because factorials grow fast: 20! still fits in a `long`, but 21! does not.

Q11. Write two overloaded `Convert` methods: one turns Celsius into Fahrenheit, the other converts a value from a given unit.

Vikash Kumar

```
Console.WriteLine($"Convert(100)          = {Temperature.Convert(100.0)} F");
Console.WriteLine($"Convert(98.6, \"F\")    = {Temperature.Convert(98.6,
"F"):F1} C");
Console.WriteLine($"Convert(37, \"C\")      = {Temperature.Convert(37.0,
"C"):F1} F");

static class Temperature
{
    public static double Convert(double celsius)
    {
        return celsius * 9 / 5 + 32;
    }

    public static double Convert(double value, string from)
    {
        if (from == "F")
        {
            return (value - 32) * 5 / 9;
        }
        return value * 9 / 5 + 32;
    }
}
```

```
Convert(100)          = 212 F
Convert(98.6, "F")    = 37.0 C
Convert(37, "C")      = 98.6 F
```

The methods are in a `static class` because top-level methods cannot be overloaded (section 7.8).

Q12. Write a method that swaps two numbers using `ref`.

```
Console.Write("First number : ");
int first = int.Parse(Console.ReadLine() ?? "");
Console.Write("Second number: ");
int second = int.Parse(Console.ReadLine() ?? "");

Swap(ref first, ref second);
Console.WriteLine($"After swap: first = {first}, second = {second}");

void Swap(ref int a, ref int b)
{
    (a, b) = (b, a);
}
```

Vikash Kumar

```
First number : 15
Second number: 40
After swap: first = 40, second = 15
```

The line `(a, b) = (b, a);` swaps the two values without a temporary variable. It is a shorter form of the three lines used in Program 7.8.

Q13. Read the marks of three subjects (0 to 100, ask again if wrong) and print the total, the percentage and the grade. Use methods.

```
int[] marks = new int[3];
string[] subjects = { "Maths", "Physics", "Chemistry" };

for (int i = 0; i < subjects.Length; i++)
{
    marks[i] = ReadMarks(subjects[i]);
}

int total = marks[0] + marks[1] + marks[2];
double percentage = total / 3.0;
Console.WriteLine($"Total      : {total} / 300");
Console.WriteLine($"Percentage : {percentage:F2}%");
Console.WriteLine($"Grade      : {GetGrade(percentage)}");

int ReadMarks(string subject)
{
    while (true)
    {
        Console.Write($"{subject} marks (0-100): ");
        if (int.TryParse(Console.ReadLine(), out int value) && value >= 0 &&
value <= 100)
        {
            return value;
        }
        Console.WriteLine("Invalid marks. Try again.");
    }
}

string GetGrade(double percentage)
{
    return percentage switch
    {
        >= 90 => "A+",
        >= 75 => "A",
        >= 60 => "B",
```

Virash Kumar

```
        >= 40 => "C",
        _ => "Fail"
    };
}
```

```
Maths marks (0-100): 88
Physics marks (0-100): 101
Invalid marks. Try again.
Physics marks (0-100): 92
Chemistry marks (0-100): 79
Total      : 259 / 300
Percentage : 86.33%
Grade      : A
```

Q14. Find the GCD of two numbers with a recursive method (Euclid's method), and use it to find the LCM.

```
Console.Write("First number : ");
int a = int.Parse(Console.ReadLine() ?? "");
Console.Write("Second number: ");
int b = int.Parse(Console.ReadLine() ?? "");

Console.WriteLine($"GCD of {a} and {b} = {Gcd(a, b)}");
Console.WriteLine($"LCM of {a} and {b} = {a / Gcd(a, b) * b}");

int Gcd(int x, int y)
{
    if (y == 0)
    {
        return x;
    }
    return Gcd(y, x % y);
}
```

```
First number : 48
Second number: 36
GCD of 48 and 36 = 12
LCM of 48 and 36 = 144
```

Euclid's idea: the GCD of `x` and `y` is the same as the GCD of `y` and the remainder `x % y`. The base case is `y == 0`, when `x` is the answer. The LCM is the product divided by the GCD; dividing first (`a / Gcd(a, b) * b`) keeps the number smaller.

Vikash Kumar

7.14 Multiple choice questions and viva

Multiple choice questions (answers at the end of the list)

1. Which keyword shows that a method returns no value?
   - (a) `null` (b) `void` (c) `empty` (d) `none`
2. The values written in the brackets of a method **call** are called:
   - (a) arguments (b) return types (c) parameters of the definition (d) overloads
3. What does the `return` statement do?
   - (a) prints a value (b) repeats the method (c) ends the method and sends a value back (d) declares a variable
4. Which one is a valid overload of `int Add(int a, int b)`?
   - (a) `double Add(int a, int b)` (b) `int Add(int x, int y)` (c) `int Add(int a, int b, int c)` (d) `void Add(int a, int b)`
5. What is printed?

```
void F(int x) { x = 10; }
int n = 5;
F(n);
Console.WriteLine(n);
```

   - (a) 0 (b) 5 (c) 10 (d) a compile-time error
6. Which parameter kind **must** be assigned by the method before it returns?
   - (a) `ref` (b) normal (c) `out` (d) optional
7. The compiler message CS0161 means:
   - (a) a variable is not declared (b) not all code paths return a value (c) wrong number of arguments (d) a missing semicolon
8. The usual naming style for a method is:
   - (a) camelCase (b) snake_case (c) UPPER_CASE (d) PascalCase
9. A recursive method must have:
   - (a) a `ref` parameter (b) a `for` loop (c) a base case (d) a `void` return type
10. Which declaration is valid?
    - (a) `void F(int a = 1, int b)` (b) `void F(int a, int b = 1)` (c) `void F(int a = 1; int b = 2)` (d) `void F(a = 1, b = 2)`

Answers: 1 (b), 2 (a), 3 (c), 4 (c; (a) and (d) differ only in the return type, and (b) differs only in parameter names), 5 (b), 6 (c), 7 (b), 8 (d), 9 (c), 10 (b).

Viva questions (answer in your own words, then compare with the chapter)

1. What is a method, and why do we divide a program into methods?

Vikash Kumar

2. What is the difference between a parameter and an argument?
3. What does pass by value mean? Show it with a small example.
4. What is the difference between `ref` and `out`? Where have you already seen `out`?
5. What is method overloading? Why could we not overload methods directly in the top-level part of the program?
6. What is recursion? What are a base case and a stack overflow?
7. Why is it better for `CalculateBill` to return the amount than to print it?

Hint for question 7: a method that only calculates can be reused in any program, can be tested by calling it with a known input, and leaves the decision of what to show to the caller.

7.15 Summary and what comes next

- A **method** is a named block of code that does one job. Define it once, call it as often as needed. Use PascalCase verb names such as `CalculateBill`.
- A **parameter** is the variable in the definition; an **argument** is the value at the call. Arguments are matched to parameters by position, or by name (`hostel: true`).
- A **return type** (`int`, `bool`, `string`, `decimal`) says what the method gives back. `void` means nothing. `return` ends the method and sends the value back; every path of a non-void method must end in a `return`.
- Give each method **one job**. Methods that calculate should not read or print, so that they can be reused and tested.
- Parameters are **copies** (pass by value), and variables declared in a method are **local** to it. Get a changed value back through the return value, or with `ref` (change the caller's variable) or `out` (send back an extra result).
- **Optional parameters** have a default value and come after the required ones.
- **Overloading** means the same name with different parameters. In these programs, overloaded methods go in a `static class`, because top-level methods cannot share a name.
- **Recursion** is a method that calls itself. It needs a base case, and each call must move closer to it; otherwise the program ends with a stack overflow.
- The compiler catches the common method mistakes: CS0161 (a path without `return`), CS7036 (missing argument), CS1503 (wrong argument type), CS0127 and CS0029 (misuse of `void`), CS0103 (variable outside its method), CS0177 (`out` not assigned), CS1620 (missing `ref`) and CS0128 (same name twice).

Vikash Kumar

Next chapter: Chapter 8, Arrays, shows how to store many values of the same kind under one name, such as the marks of 60 students. You will use loops to fill and process arrays, pass arrays to methods, and build two-dimensional arrays for tables like a timetable.

Vikash Kumar